Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ Thông tin 1

Nhập môn học sâu

Perceptron đa tầng

TS. NGUYỄN XUÂN ĐỨC

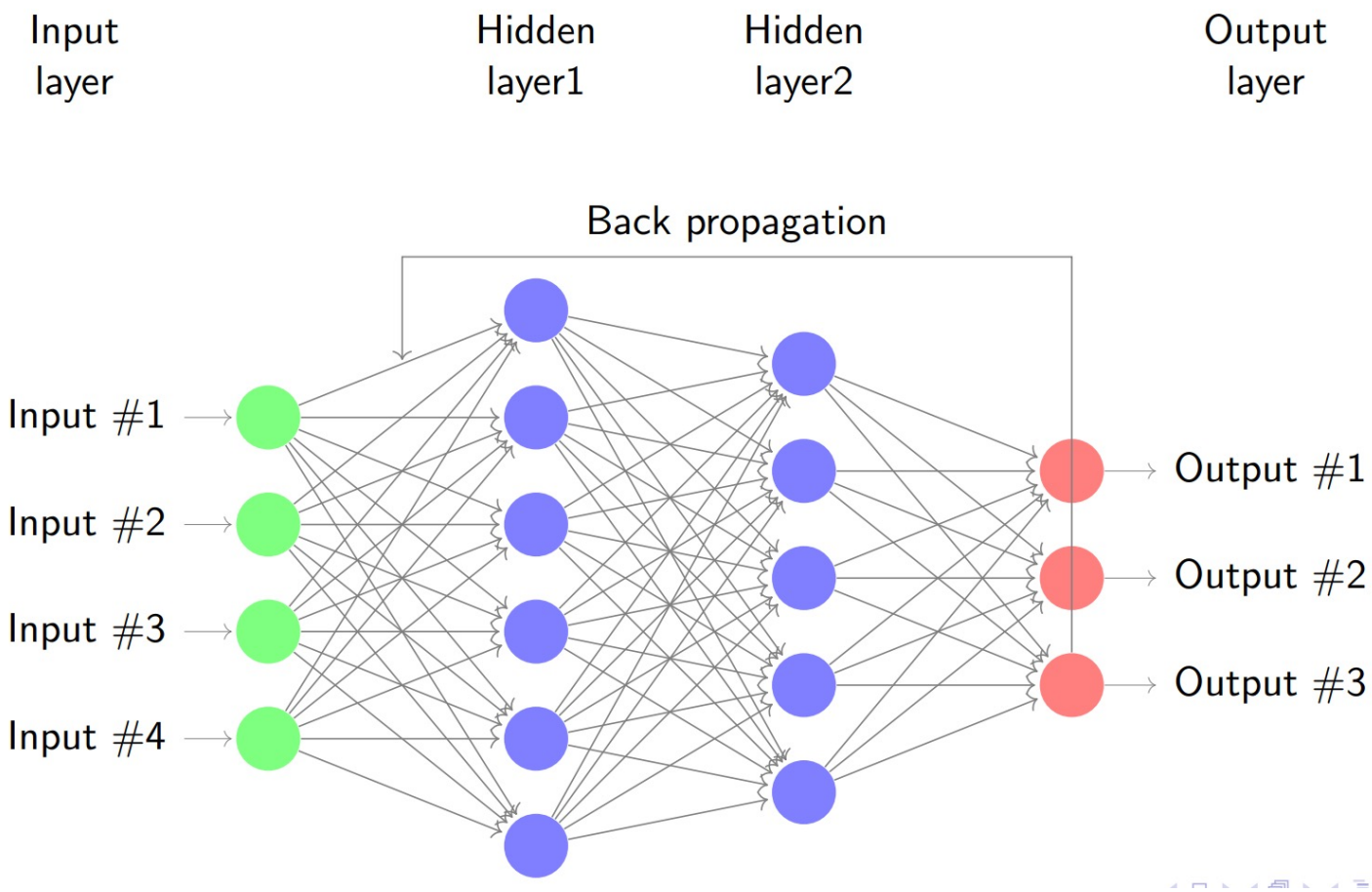
Nội dung

- Perceptron đa tầng
- Lan truyền xuôi
- Lan truyền ngược
- Điều chuẩn (regularization)

Giới hạn của mô hình tuyến tính

- Giả định về tính đơn điệu: giá trị đặc trưng tăng (giảm) dẫn đến giá trị đầu ra của mô hình tăng (giảm)
  - Hợp lý trong một số tình huống: thu nhập tăng -> tăng khả năng trả nợ vay
  - Phản ví dụ: phân loại ảnh chó mèo. Giá trị điểm ảnh tăng -> tăng khả năng bức ảnh thuộc lớp chó (mèo)?
- Giải pháp: thêm tầng ẩn và hàm kích hoạt

Ví dụ MLP


Input
layer
Hidden
layer1
Hidden
layer2
Output
layer
Back propagation
Input #1
Input #2
Input #3
Input #4
Output #1
Output #2
Output #3

Perceptron đa tầng (MLP)

- MLP: mô hình $y(.,w)$ ánh xạ đầu vào D-chiều sang đầu ra C-chiều

$$y(.,w): R^D \to R^C$$

$$x \mapsto y(x,w) = \begin{pmatrix} y_1(x,w) \\ y_C(x,w) \end{pmatrix}$$

- Trong mô hình MLP với (L + 1) tầng, mỗi tầng l là tổ hợp tuyến tính của tầng $(l-1)$ (hoặc đầu vào)

MLP – Tầng thứ nhất

- Với đầu vào $x \in R^D$, tầng $l = 1$ tính vector $y^{(1)} = \left(y_1^{(1)}, \dots, y_{N^{(1)}}^{(1)}\right)$:

$$y_i^{(1)} = f\left(z_i^{(1)}\right) \text{ với } z_i^{(1)} = \sum_{j=1}^{D} w_{i,j}^{(1)} x_j + b_i^{(1)}$$

với f là hàm kích hoạt và $w_{i,j}^{(1)}, b_i^{(1)}$ là trọng số học được

- Vector hoá:

$$y^{(1)} = f\left(W^{(1)} x + b^{(1)}\right)$$

MLP – Tầng sâu hơn

- Tổng quát, tầng l tính vector $y^{(l)} = \left(y_1^{(l)}, \dots y_{N^{(l)}}^{(l)}\right)$:

$$y^{(l)} = f\left(W^{(l)} y^{(l-1)} + b^{(l)}\right)$$

- Hàm kích hoạt giữa các lớp có thể khác nhau

MLP – Tầng đầu ra

- Tầng $(L+1)$ được gọi là tầng đầu ra:

$$\hat{y} = f\left(W^{(l+1)}y^{(l)} + b^{(l+1)}\right)$$

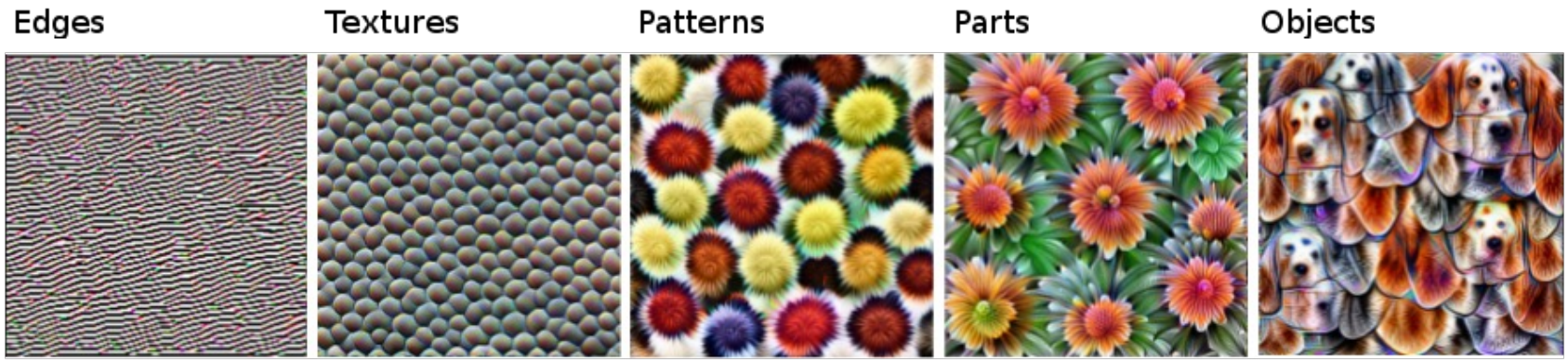
trong đó $\hat{y} \in R^C$

- Với bài toán phân lớp: $\hat{y} = softmax\left(W^{(l+1)}y^{(l)} + b^{(l+1)}\right)$

trong đó $softmax(o_i) = \frac{exp(o_i)}{\sum_j exp(o_j)}$

Tại sao mạng nơ ron có nhiều tầng

- Học các đặc trưng ở mức độ trừu tượng hóa khác nhau



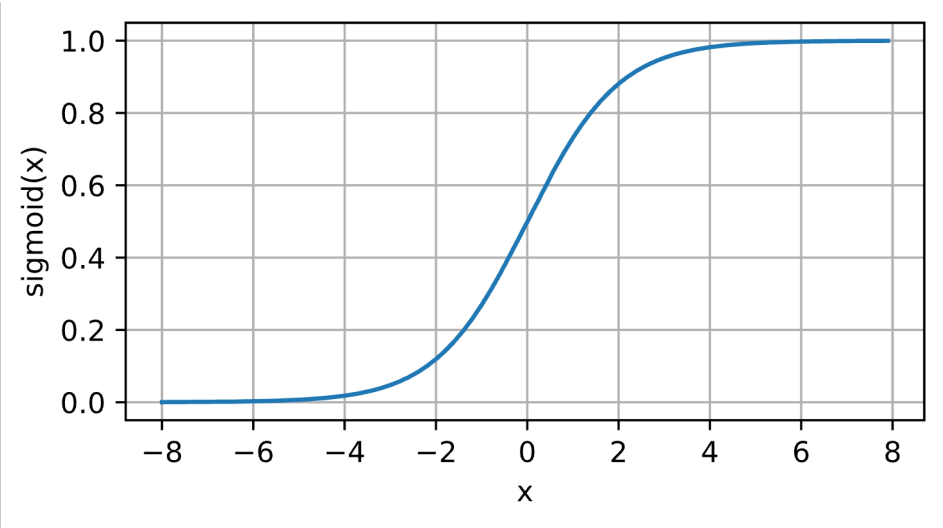
Đặc trưng học bởi CNN (Inception V1) huấn luyện trên tập ImageNet

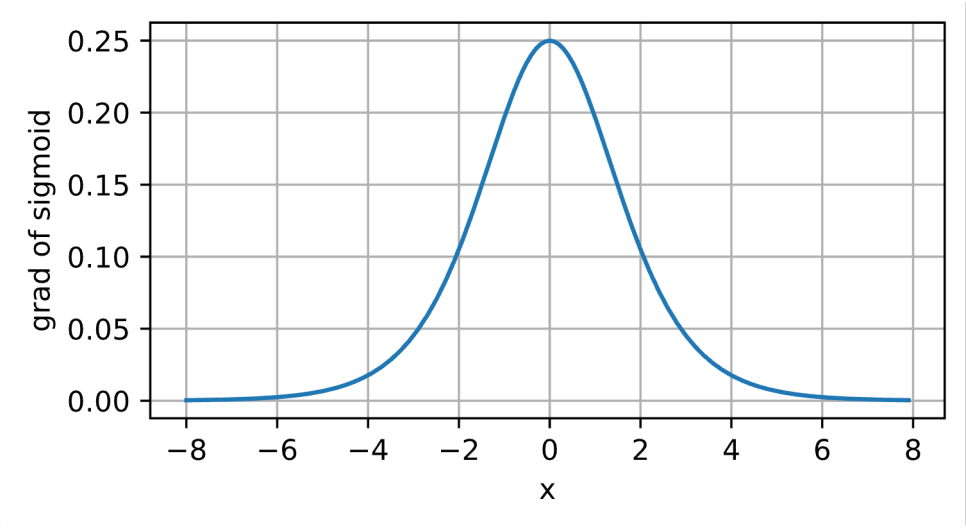
Hàm kích hoạt

- Hàm kích hoạt: xác định một nơ ron có được kích hoạt hay không
- Định lý xấp xỉ (universal approximation theorem): mạng nơ ron với một tầng ẩn có thể học được hàm số bất kỳ

Hàm sigmoid

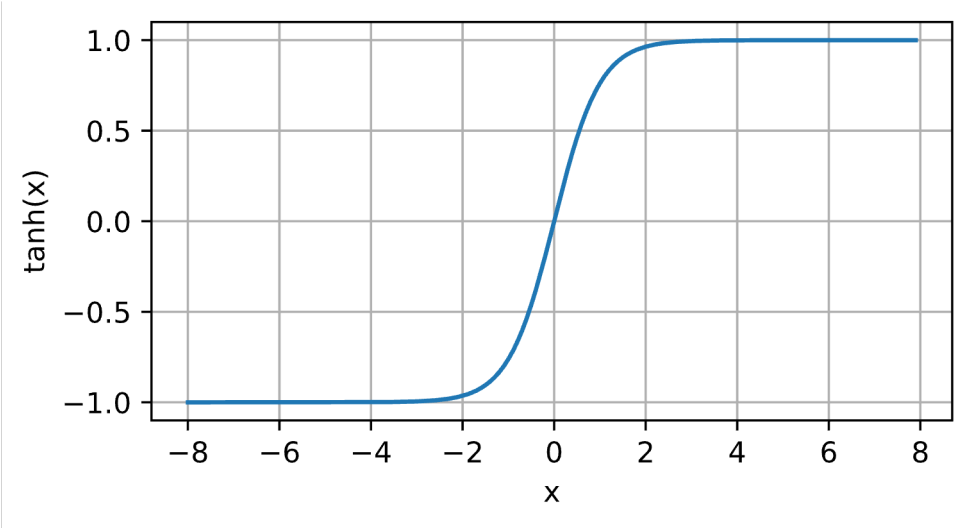
- Công thức: $sigmoid(x) = \frac{1}{1+exp(-x)}$
- Đạo hàm: $\frac{d}{dx} sigmoid(x) = \frac{exp(-x)}{(1+exp(-x))^2} = sigmoid(x)(1 - sigmoid(x))$

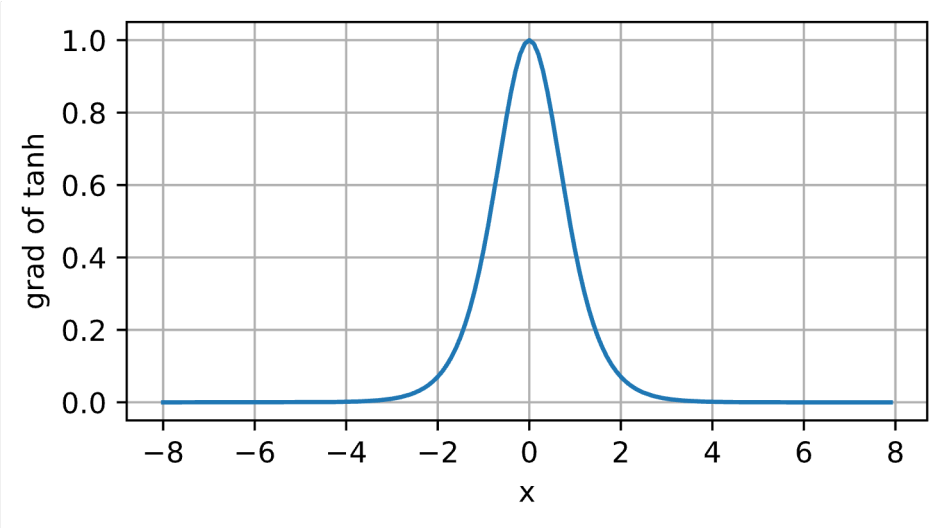




Hàm tanh

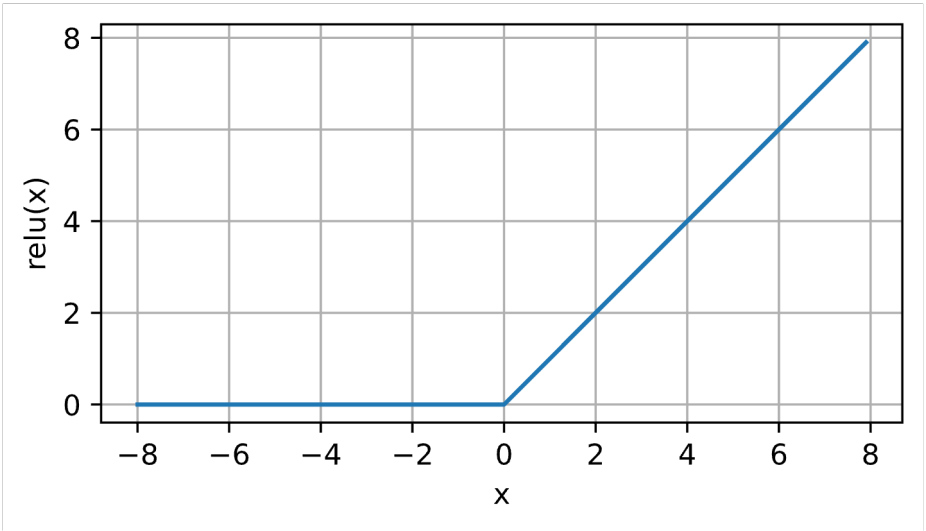
- Công thức: $tanh(x) = \frac{1-exp(-2x)}{1+exp(-2x)}$
- Đạo hàm: $\frac{d}{dx} tanh(x) = 1 - tanh^2(x)$

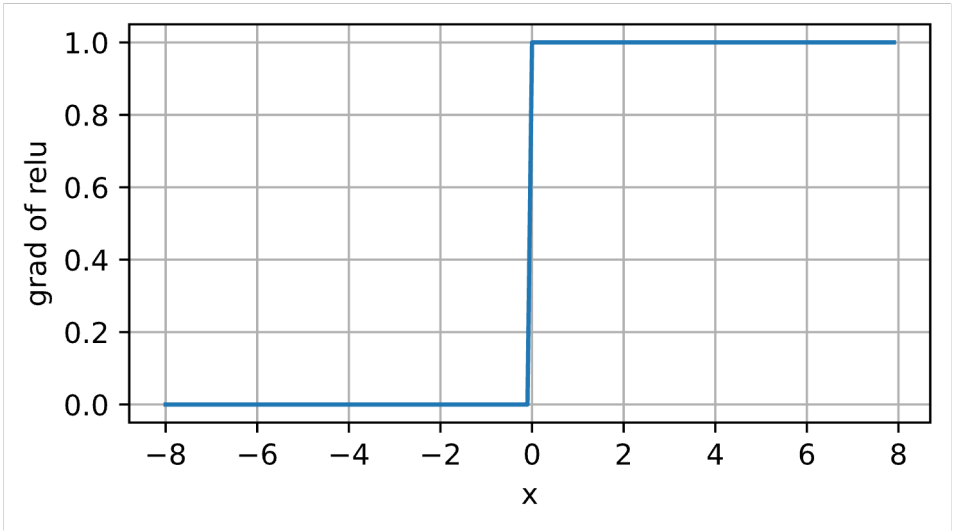




Hàm ReLU

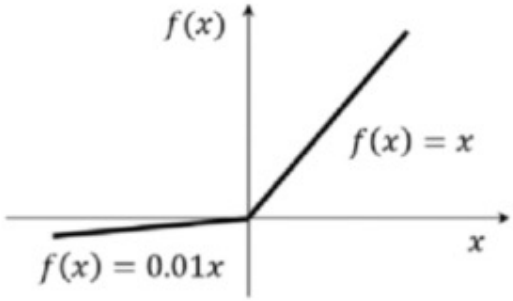
- Công thức: $ReLU(x) = max(x, 0)$
- Tính chất:
  - Tính đạo hàm đơn giản
  - Giảm thiểu nhược điểm tiêu biến gradient



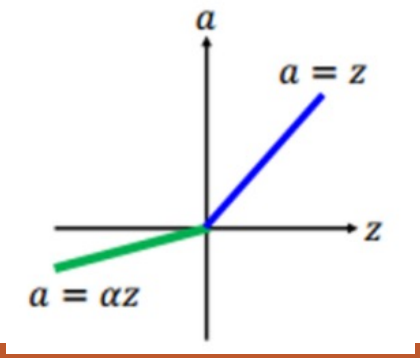


Biến thể của ReLU

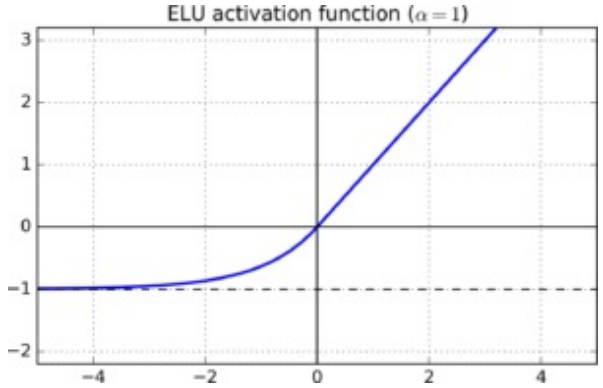
- LeakyReLU: $f(x) = \max(0.01x, x)$



- Parametric ReLU: $f(x) = \max(\alpha x, x)$



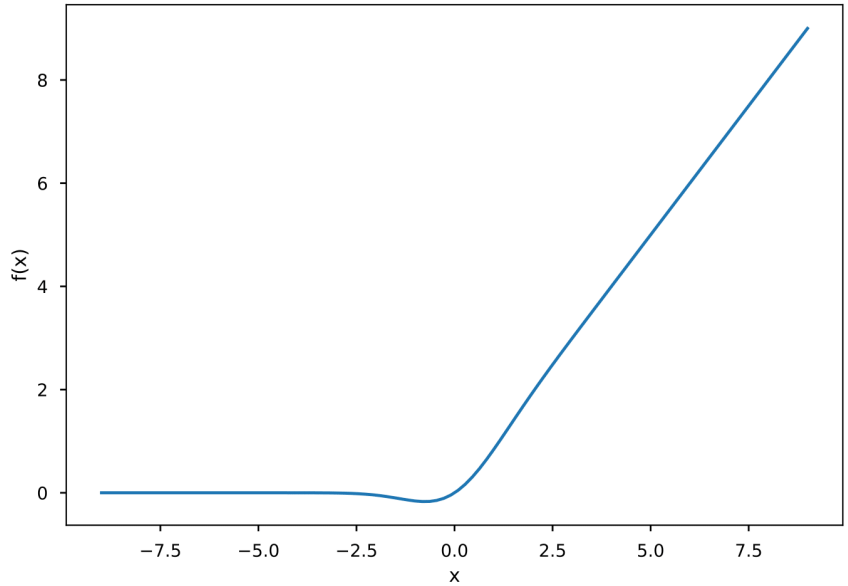
- ELU: $\begin{cases} x \; for \; x \geq 0 \\ \alpha(e^x - 1) \; for \; x < 0 \end{cases}$

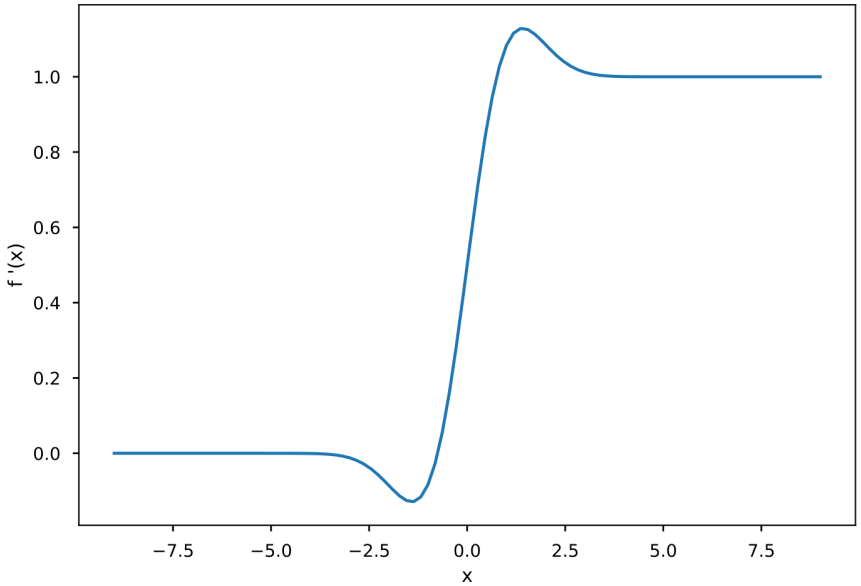


Hàm GELU

- Công thức: GELU(x) = $x \times \Phi(x)$

trong đó $\Phi(x)$ là hàm phân phối tích lũy của phân phối Gaussian





Huấn luyện mạng nơ ron

- Chúng ta có mô hình $\mathrm{f}(.,w)$ với trọng số học được W
- Yêu cầu: xác định trọng số để giải bài toán
  - Hồi quy
  - Phân lớp

Tập huấn luyện

- Cho tập huấn luyện

$$U_s = \{(x_n, y_n): 1 \leq n \leq N\}$$

với hàm mục tiêu bình phương

$$E(w) = \sum_{n=1}^{N} E_n(w) = \sum_{n=1}^{N} \sum_{i=1}^{C} \left(y_i(x_n, w) - y_{n,i}\right)^2$$

Tối ưu mạng nơ ron

- Stochastic Training: mẫu (x_n, t_n) được chọn ngẫu nhiên, w được cập nhật để cực tiểu hóa $E_n(w)$
- Minibatch Training: một tập M mẫu được chọn ngẫu nhiên và trong số w được cập nhật dựa trên lỗi $E_M(w) = \frac{1}{|M|}\sum_{n \in M} E_n(w)$

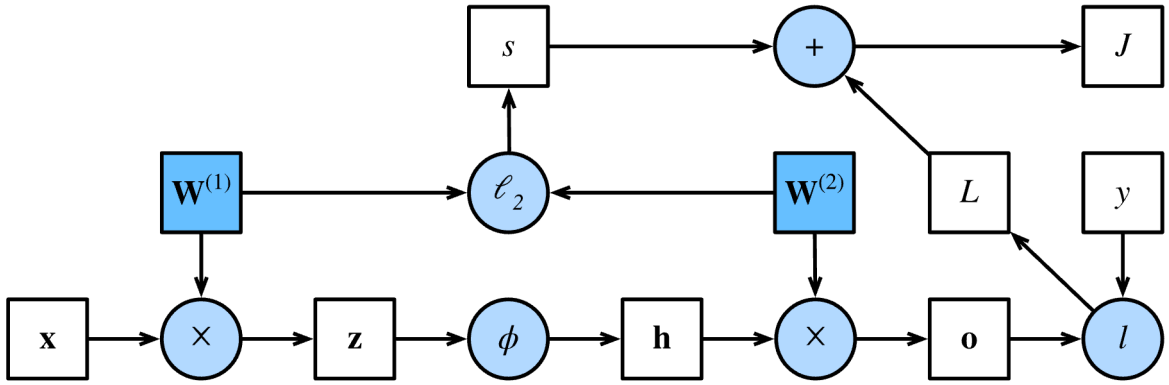
Tối ưu tuần tự

- Câu hỏi: làm sao để cực tiểu hóa $E_n(w)$?
  - $E_n(w)$ là hàm phi tuyến với nhiều cực trị địa phương
- Giải pháp: gradient descent
  - Gọi $w[0]$ là giá trị ban đầu của trọng số
  - $w[t]$ là trọng số tại bước t
  - Tại bước $t+1$, ta cập nhật trọng số:

$$w[t+1] = w[t] - \alpha \frac{\partial J}{\partial w[t]}$$

Lan truyền xuôi

- Xét mạng nơ ron với một tầng ẩn, gọi $x \in R^D$ là một mẫu huấn luyện
- Đầu ra của tầng ẩn : $h = \phi\left(W^{(1)}x\right)$ trong đó $W^{(1)} \in R^{h \times d}$ và ϕ là hàm kích hoạt
- Đầu ra của mạng nơ ron : $o = W^{(2)}h$
- Hàm mục tiêu: $J = l(o, y) + \frac{\lambda}{2}\left(\left\|W^{(1)}\right\|_F^2 + \left\|W^{(2)}\right\|_F^2\right)$



Đồ thị tính toán

Lan truyền ngược

- Định nghĩa: phương pháp tính gradient bằng cách duyệt qua mạng nơ ron theo chiều ngược lại, từ đầu ra đến đầu vào, sử dụng quy tắc dây chuyền để tính gradient
- Gradient của hàm mục tiêu J theo hàm mất mát L và điều chuẩn s:

$$\frac{\partial J}{\partial L} = 1 \text{ và} \frac{\partial J}{\partial s} = 1$$

- Gradient của hàm mục tiêu theo các biến của lớp đầu ra :

$$\frac{\partial J}{\partial o} = \frac{\partial J}{\partial L}\frac{\partial L}{\partial o} = \frac{\partial L}{\partial o} = o - y$$

- Gradient của điều chuẩn theo tham số :

$$\frac{\partial s}{\partial W^{(1)}} = \lambda W^{(1)} \text{ và} \frac{\partial s}{\partial W^{(2)}} = \lambda W^{(2)}$$

- Gradient $\frac{\partial J}{\partial W^{(2)}} = \frac{\partial J}{\partial o}\frac{\partial o}{\partial W^{(2)}} + \frac{\partial J}{\partial s}\frac{\partial s}{\partial W^{(2)}} = \frac{\partial J}{\partial o} h^{\mathrm{T}} + \lambda W^{(2)}$

Backpropagation

- Gradient theo đầu ra tầng ẩn :

$$\frac{\partial J}{\partial h} = \frac{\partial J}{\partial o}\frac{\partial o}{\partial h} = W^{(2)\mathrm{T}}\frac{\partial J}{\partial o}$$

- Gradient theo biến trung gian z:

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial h}\frac{\partial h}{\partial z} = \frac{\partial J}{\partial h} \odot \phi'(z)$$

- Gradient theo tham số $W^{(1)}$:

$$\frac{\partial J}{\partial W^{(1)}} = \frac{\partial J}{\partial z}\frac{\partial z}{\partial W^{(1)}} + \frac{\partial J}{\partial s}\frac{\partial s}{\partial W^{(1)}} = \frac{\partial J}{\partial z} x^{\mathrm{T}} + \lambda W^{(1)}$$

Ổn định số học

- Xét một mạng nơ ron sâu với L tầng, đầu vào x và đầu ra o. Mỗi tầng l được định nghĩa bởi phép biến đổi f_l với tham số W_l. Mạng nơ ron được biểu diễn:

$$o = f_L \circ \cdots \circ f_1(x)$$

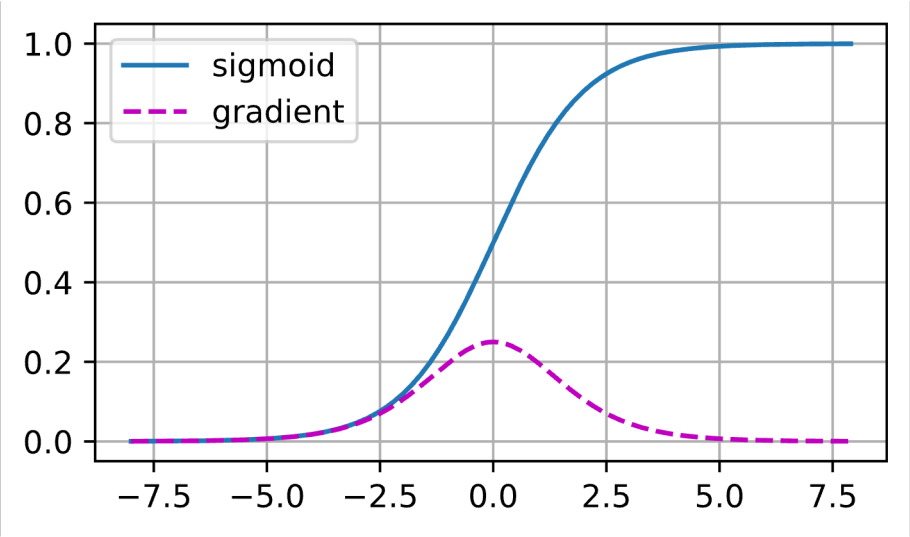
- Gradient của o theo tham số $W^{(l)}$:

$$\frac{\partial o}{\partial W^{(l)}} = \frac{\partial h^{(L)}}{\partial h^{(L-1)}} \cdots \frac{\partial h^{(l)}}{\partial W^{(l)}}$$

- Theo đó, gradient trên là tích của $L - l$ ma trận và giá trị đó có thể *quá nhỏ* (tiêu biến gradient) hoặc *quá lớn* (bùng nổ gradient)

Tiêu biến gradient

- Thông thường, vấn đề tiêu biến gradient xảy ra khi dùng hàm kích hoạt sigmoid
- Gradient của hàm sigmoid tiêu biến khi đầu vào quá nhỏ hoặc quá lớn
- Do đó hàm kích hoạt ReLU thường được sử dụng hơn



Bùng nổ gradient

- Ngược lại, tích các ma trận có thể quá lớn
- Ví dụ, nhân 100 ma trận sinh ngẫu nhiên từ phân phối Gaussian:

PYTORCH MXNET JAX TENSORFLOW

```
M = torch.normal(0, 1, size=(4, 4))
print('a single matrix \n',M)
for i in range(100):
    M = M @ torch.normal(0, 1, size=(4, 4))
print('after multiplying 100 matrices\n', M)
```

```
a single matrix
 tensor([[-0.8755, -1.2171,  1.3316,  0.1357],
        [ 0.4399,  1.4073, -1.9131, -0.4608],
        [-2.1420,  0.3643, -0.5267,  1.0277],
        [-0.1734, -0.7549,  2.3024,  1.3085]])
after multiplying 100 matrices
 tensor([[-2.9185e+23,  1.3915e+25, -1.1865e+25,  1.4354e+24],
        [ 4.9142e+23, -2.3430e+25,  1.9979e+25, -2.4169e+24],
        [ 2.6578e+23, -1.2672e+25,  1.0805e+25, -1.3072e+24],
        [-5.2223e+23,  2.4899e+25, -2.1231e+25,  2.5684e+24]])
```

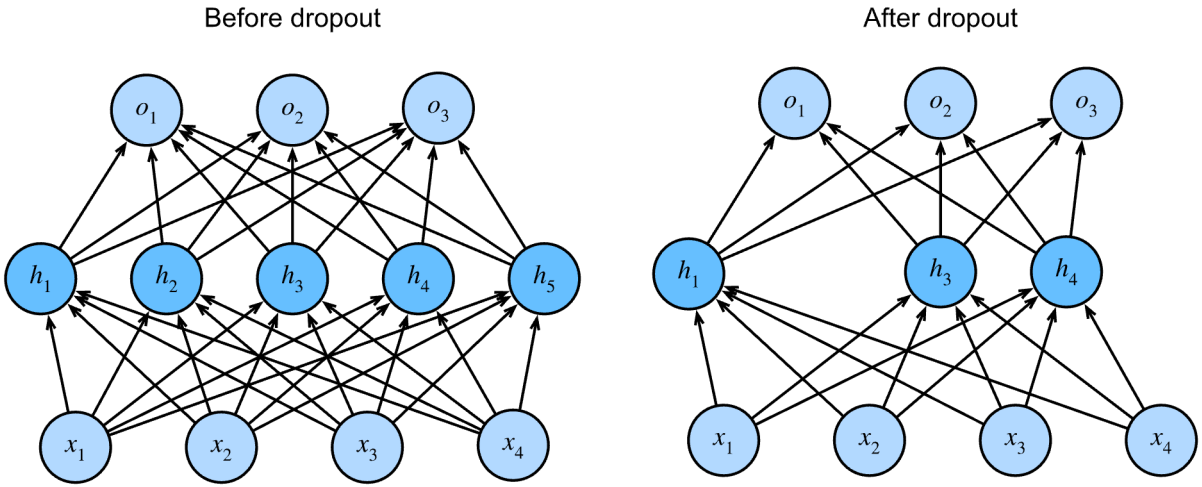
Khởi tạo tham số

- Thông thường, mô hình hoạt động tốt nếu tham số được khởi tạo ngẫu nhiên xung quanh 0
- Các cách khởi tạo tham số:
  - 0 → không nên dùng
  - Lấy ngẫu nhiên trong khoảng [0,1] theo phân phối đồng nhất → không tốt với mạng nhiều lớp
  - Xavier: lấy ngẫu nhiên từ $U\left[-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right]$

Dropout

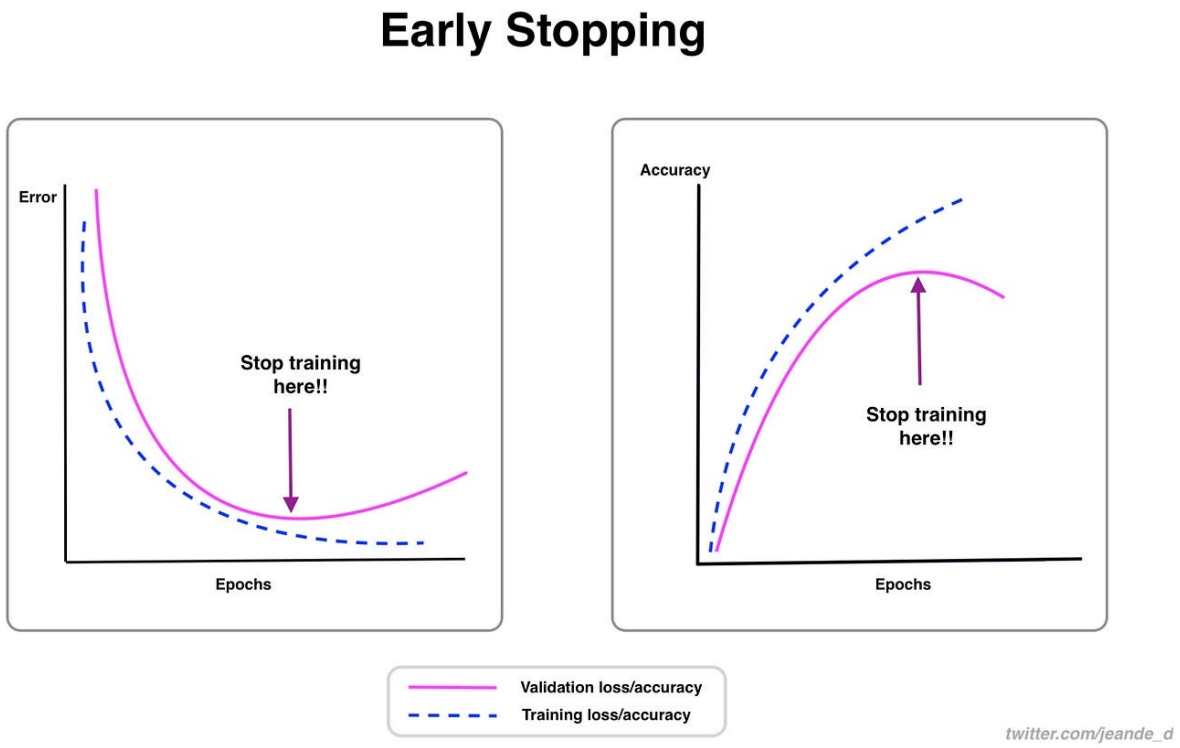
- Một mô hình tốt cần hoạt động tốt trên dữ liệu chưa biết
  - Chúng ta cần mô hình đơn giản nhất có thể

- Trong phương pháp dropout, mỗi giá trị trung gian h được thay thế với giá trị h' như sau: $h' = \begin{cases} 0. & \text{với xác suất } p \\ \frac{h}{1-p}. & \text{ngược lại} \end{cases}$



Early stopping

- Dừng huấn luyện khi hàm mất mát tăng sau một số vòng lặp (epoch) nhất định



Thực hành

- Lập trình huấn luyện mạng nơ ron với PyTorch